
Challenge — DevOps & Platform Engineering

El escenario

Son las 11am de un martes. El equipo de producto acaba de confirmar que necesitan una API para exponer métricas de infraestructura al resto de la organización. El contrato con el cliente enterprise se firma en 3 semanas.

El CTO te llama y te dice tres cosas:

"Necesito una API funcionando en producción, en cloud, con alta disponibilidad. El cliente espera 10.000 requests por segundo en pico. Y no quiero sorpresas a las 3am."

No hay equipo dedicado — sos vos. Tenés acceso a AWS o GCP, presupuesto razonable sin carta blanca, y libertad para elegir el stack.

Tu trabajo es dos cosas: **construir la API** y **llevarla a producción**.

Parte 1 — Construcción de la API

Diseñá y desarrollá una API REST (o GraphQL si lo justificás) que exponga información de infraestructura. El dominio es tuyo — podés modelar métricas de disponibilidad, eventos de deploy, alertas, estado de servicios, o lo que consideres relevante para un equipo de infra.

Lo que sí esperamos:

- Al menos **3 endpoints** con propósito claro
- **Manejo de errores** consistente
- **Health check** endpoint (lo vas a necesitar en la Parte 2)
- Containerizada — debe correr con `docker run` o `docker compose up`
- **Sin credenciales hardcodeadas** en el código

El lenguaje, framework y base de datos (si la necesitás) son decisión tuya. Justificala.

Parte 2 — Deployment a producción

Tomando la API de la Parte 1, llevarla a producción en cloud.

Arquitectura

Proponé una arquitectura en **AWS o GCP** que permita:

- Exposición pública segura
- Alta disponibilidad
- Escalabilidad (10k RPS)
- Tolerancia a fallos
- Manejo de secretos
- Monitoreo básico

Entregá un diagrama + explicación de tus decisiones.

CI/CD

Definí el pipeline — claridad en etapas, no YAML perfecto:

- Build & test
- Análisis de calidad / seguridad
- Build de imagen
- Push a registry
- Deploy automático
- Estrategia de rollback

Infrastructure as Code

Templates (Terraform idealmente) para desplegar:

- Networking básico
- Compute / runtime
- Load balancing
- IAM / seguridad
- Observabilidad mínima

Entregables

None

<code>/api</code>	→ código fuente + Dockerfile
<code>/iac</code>	→ templates IaC
<code>/cicd</code>	→ definición del pipeline

README / run.md → cómo levantar local + cómo desplegar + decisiones
prompts.md → opcional: si usaste IA, qué y cómo

Qué evaluamos

- Criterio de diseño de la API (no solo que funcione)
 - Diseño para producción real
 - Escalabilidad y resiliencia
 - Seguridad por defecto
 - Automatización end-to-end
 - Claridad en decisiones y trade-offs
-

Bonus

- Autoscaling
 - WAF / rate limiting
 - Blue/green o canary
 - Observabilidad sólida
 - Tests automatizados
 - Uso práctico de IA
-

Deseables — preguntas abiertas

No son requisitos. Si las trabajaste, las discutimos en la presentación. Si no, la pregunta igual la vamos a hacer.

Estrategia multi-ambiente — ¿cómo garantizás paridad staging/prod sin procesar datos sensibles en staging?

Observabilidad con criterio — SLOs/SLIs, estrategia de alertas, distributed tracing. No solo nombrar herramientas.

Costo como variable de diseño — ¿podés justificar el costo estimado? ¿Qué tradeoffs tomaste?

Governance y cambios en producción — ¿cómo gestionarías un cambio crítico en prod fuera de horario?

La presentación

Los primeros minutos son tuyos — contanos qué construiste y por qué. Después vamos directo a las decisiones: las que tomaste y las que no.

No hay una única solución correcta. Hay decisiones bien y mal fundamentadas.
